

In the Name of God



Course Project: Introduction to Machine Learning

Phase 1

Out-of-Distribution Detection with GMM and EM

Instructor

Dr. S. Amini

Department of Electrical Engineering

Sharif University of Technology

Fall 1404

Deadline:

13 Day, 1404 at 23:55

Authors:

Ashkan Yousefnia, Arshak Razvani, Zahra Sorkhaei, Rojin Salmani,
Mohammad Eshtehardian, Mohammad Hossein Momeni

Contents

Contents

1	Important Notes	1
2	Out-of-Distribution (OOD): What it is, why it is useful, and how it is used	2
2.1	What is OOD?	2
2.2	Why is OOD detection useful?	2
2.3	How is OOD detection used in practice?	2
3	Different Approaches to OOD Detection	3
4	Proposed Method in this Project: Modeling the ID Distribution with a GMM	4
4.1	Why Density-Based OOD Detection?	4
4.2	Likelihood as an OOD Score	4
4.3	Modeling the ID Density with a Gaussian Mixture Model (GMM)	4
4.4	OOD Decision with the Trained GMM	4
4.5	Remark on Likelihood Pitfalls	5
5	Training with EM	5
5.1	Maximum Likelihood Objective	5
5.2	Expectation–Maximization (EM)	5
5.3	Initialization	5
5.4	E-step	5
5.5	M-step	6
5.6	Convergence and Practical Considerations	6
5.7	How OOD Samples are Classified (Likelihood Thresholding)	6

1 Important Notes

Please carefully read the following points:

1. The two phases of this project will contribute a total of **4 points** to your final course grade.
2. After the final submission, there will be an in-person project defense session where you must present your code and outputs to the teaching assistants and answer their questions. **All group members must be fully familiar with every part of the project.** A single score will be assigned to the entire group.
3. All simulations must be implemented using Python. You are allowed to use any libraries you have used in the assignments, such as `numpy`, `scipy`, and `pytorch`. However, **you must implement the algorithms yourself**, and you may not use libraries that already provide the algorithm implementations.
4. The project deliverables consist of both your code and a written report. The report should include answers to the questions, plots, figures, and conclusions. In the end, one `.zip` file containing all code files, and a `.pdf` report must be uploaded to the CW platform. Only one group member needs to submit the project.
5. If you use any external sources (books, articles, websites, etc.) to answer the questions, make sure to cite them properly.
6. **Any form of plagiarism will result in a zero for both parties involved.**
7. For any questions, please contact the project instructors via: on Telegram.

Good luck!

2 Out-of-Distribution (OOD): What it is, why it is useful, and how it is used

2.1 What is OOD?

Most machine learning models are trained under an i.i.d. assumption where training and test data are drawn from the same distribution (the *in-distribution*, ID). In real deployments (open-world settings), the system may face inputs that do not follow the training distribution; these are called *out-of-distribution* (OOD) samples.

Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ be the label space, with a joint distribution $P(X, Y)$ over $\mathcal{X} \times \mathcal{Y}$. Distribution shift can be broadly described as:

- **Covariate shift:** the marginal input distribution changes, i.e., $P(X) \neq P'(X)$ (typically assuming the same label space).
- **Semantic (label) shift:** the semantic labels differ across training and testing, i.e., $P(Y) \neq P'(Y)$, which often also implies changes in $P(X)$.

In OOD detection for classification, the common focus is **semantic shift**: test samples can come from classes/semantics outside the support of the training label space, and the model should *reject* them instead of forcing an in-label prediction. A compact formalization is:

$$\text{ID: } (X, Y) \sim P(X, Y), \quad \text{OOD: } (X, Y) \sim P'(X, Y), \quad P(Y) \neq P'(Y). \quad (1)$$

2.2 Why is OOD detection useful?

Modern models (especially deep networks) can be overconfident on inputs that are far from the training distribution, which can be harmful in real systems. Therefore, a practical recognition system should be accurate on ID inputs and also:

- **Detect unknown/novel inputs** and **reject** them, or
- **Defer** them to a human or a safer fallback pipeline.

The main emphasis is on safety-critical settings (e.g., autonomous driving) where unusual scenes/objects should trigger warnings or handover, and it also motivates OOD detection with everyday examples such as a food classifier that should reject non-food images.

2.3 How is OOD detection used in practice?

A standard deployment pattern is: train on ID data, then at test time compute an *OOD score* (or *ID-ness score*) $s(x)$ for each input x and compare it to a threshold τ :

$$\delta(x) = \begin{cases} \text{ID} & s(x) \geq \tau, \\ \text{OOD} & s(x) < \tau. \end{cases} \quad (2)$$

If $\delta(x) = \text{ID}$, the model outputs a standard prediction $\hat{y} = f(x)$; otherwise the sample is rejected/flagged for a fallback route.

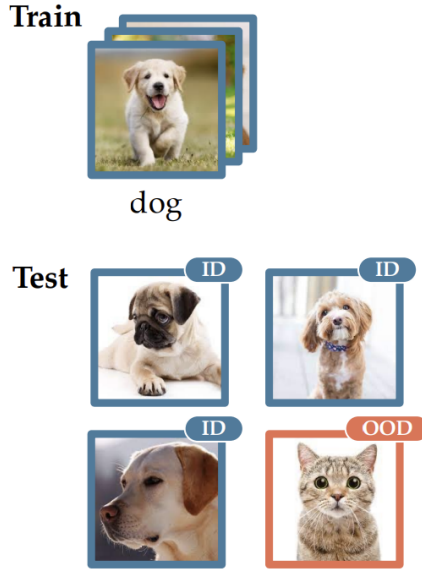


Figure 1: A simple example demonstrating OOD detection.

3 Different Approaches to OOD Detection

Most OOD detectors produce an *OOD score* $s(\mathbf{x})$ for a test input $\mathbf{x}$ and then *reject* it as OOD if $s(\mathbf{x})$ indicates low “in-distribution-ness” (e.g., below/above a threshold).

- **Classification-based methods.** Train a classifier on in-distribution (ID) classes, then derive an OOD score from the classifier behavior—most commonly from its output confidence (e.g., softmax-based confidence / logit-based scoring) without necessarily changing the training procedure. *ID inputs should look confident/consistent under the classifier, while OOD inputs should look uncertain or inconsistent.*
- **Density-based methods.** Explicitly model the ID data distribution with a probabilistic density model and flag test samples that fall in *low-density* (low-likelihood) regions as OOD. *OOD inputs are unlikely under the learned ID distribution* (optionally using likelihood-ratio style scores to reduce known likelihood pitfalls).
- **Distance-based methods.** Map data to a feature/embedding space (often from a classifier) and measure how far a test point is from ID *prototypes*, *centroids*, or *neighbors*. *OOD inputs lie far from the ID feature clusters*, so large distances indicate OOD.
- **Reconstruction-based methods.** Train an encoder–decoder (e.g., autoencoder-style) on ID data and use the fact that ID and OOD typically yield different reconstruction behavior. *models trained on ID reconstruct ID well but fail (or behave differently) on OOD*, so reconstruction error or reconstruction inconsistency becomes the OOD score.

4 Proposed Method in this Project: Modeling the ID Distribution with a GMM

4.1 Why Density-Based OOD Detection?

In this project, we choose a *density-based* approach for out-of-distribution (OOD) detection. The main idea is to explicitly model the in-distribution (ID) data by learning a probability density $p_{\text{ID}}(\mathbf{x})$, and then use the (log-)likelihood of a test sample under this ID model as an OOD score. Samples that lie in *low-density* (low-likelihood) regions of the learned ID distribution are considered OOD. This follows the general viewpoint of density-based OOD detection discussed in the previous part.

4.2 Likelihood as an OOD Score

Given a learned ID density $p_{\text{ID}}(\mathbf{x})$, a natural OOD score is the log-likelihood

$$s(\mathbf{x}) = \log p_{\text{ID}}(\mathbf{x}). \quad (3)$$

Then OOD detection is performed by thresholding:

$$\delta(\mathbf{x}) = \begin{cases} \text{ID} & s(\mathbf{x}) \geq \tau, \\ \text{OOD} & s(\mathbf{x}) < \tau, \end{cases} \quad (4)$$

where τ is chosen using a validation procedure (e.g., controlling the false positive rate on ID validation data). In words: *if a test sample is very unlikely under the learned ID distribution, we reject it as OOD.*

4.3 Modeling the ID Density with a Gaussian Mixture Model (GMM)

To realize $p_{\text{ID}}(\mathbf{x})$, we model the ID distribution using a Gaussian Mixture Model (GMM), which can represent multi-modal densities as a weighted sum of Gaussian components:

$$p_{\text{ID}}(\mathbf{x}; \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad (5)$$

subject to

$$\pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1. \quad (6)$$

Here, K is the number of mixture components, π_k are mixture weights, and each component has mean $\boldsymbol{\mu}_k$ and covariance $\boldsymbol{\Sigma}_k$. We denote the parameters by

$$\Theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K. \quad (7)$$

4.4 OOD Decision with the Trained GMM

After fitting the GMM on ID data, we compute the log-likelihood score

$$s(\mathbf{x}) = \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right), \quad (8)$$

and apply the threshold rule in Eq. (4). Optionally, for interpretability, we can also compute the posterior responsibility of each component

$$\gamma_k(\mathbf{x}) = p(z = k \mid \mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \quad (9)$$

4.5 Remark on Likelihood Pitfalls

Note that pure likelihood scoring can sometimes behave unexpectedly (certain OOD samples may receive high likelihood), and discusses more robust variants such as likelihood-ratio style scoring. In this phase, we start with the standard likelihood thresholding in Eq. (4) and consider extensions if needed.

5 Training with EM

5.1 Maximum Likelihood Objective

Given ID training samples $\{\mathbf{x}_i\}_{i=1}^n$, we fit the GMM parameters $\Psi = \{(\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\}_{k=1}^K$ by maximizing the likelihood of the observed data. The (joint) likelihood can be written as

$$L(\Psi) = \prod_{i=1}^n f(\mathbf{x}_i; \Psi), \quad (10)$$

and it is more convenient to maximize the log-likelihood $\log L(\Psi)$.

5.2 Expectation–Maximization (EM)

Direct maximization of the mixture log-likelihood is difficult due to the latent component assignments. The EM algorithm provides an iterative procedure to compute maximum-likelihood estimates for mixture models by alternating between: (i) estimating the latent assignments under current parameters (E-step), and (ii) updating parameters to improve likelihood (M-step).

5.3 Initialization

We start from an initial parameter set $\Psi^{(0)}$. A common practice is to use k -means to initialize the component means (and then initialize covariances and weights from the induced clusters).

5.4 E-step

Introduce a latent variable $z_i \in \{1, \dots, K\}$ indicating which Gaussian generated $\mathbf{x}_i$. At iteration t , compute the posterior responsibility of component k for sample i :

$$\gamma_{ik}^{(t)} \triangleq p(z_i = k \mid \mathbf{x}_i; \Psi^{(t)}) = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}. \quad (11)$$

Conceptually, this is the “conditional expectation” of component membership given current parameters.

5.5 M-step

Using responsibilities, define the effective membership counts:

$$N_k^{(t)} = \sum_{i=1}^n \gamma_{ik}^{(t)}. \quad (12)$$

Update mixture weights, means, and covariances:

$$\pi_k^{(t+1)} = \frac{N_k^{(t)}}{n}, \quad (13)$$

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k^{(t)}} \sum_{i=1}^n \gamma_{ik}^{(t)} \mathbf{x}_i, \quad (14)$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k^{(t)}} \sum_{i=1}^n \gamma_{ik}^{(t)} \left(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)} \right) \left(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)} \right)^\top. \quad (15)$$

This step updates Ψ to increase (or at least not decrease) the data likelihood.

5.6 Convergence and Practical Considerations

Steps E and M are repeated until the likelihood stabilizes (converges to a local optimum). In practice, convergence is often checked by either: (i) a maximum iteration limit (e.g., 100 iterations), or (ii) a small change threshold between iterations (e.g., 10^{-3}).

Since EM can be sensitive to initialization, a standard strategy is to use k -means centroids for the initial means and/or run EM multiple times with different random initializations and keep the best solution.

5.7 How OOD Samples are Classified (Likelihood Thresholding)

After training, we obtain an estimate of the ID density $p_{\text{ID}}(\mathbf{x}; \Psi^*)$. For a test sample $\mathbf{x}$, we compute its (log-)likelihood under the trained GMM:

$$s(\mathbf{x}) = \log p_{\text{ID}}(\mathbf{x}; \Psi^*) = \log \left(\sum_{k=1}^K \pi_k^* \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k^*, \boldsymbol{\Sigma}_k^*) \right). \quad (16)$$

We then classify $\mathbf{x}$ using a threshold τ :

$$\delta(\mathbf{x}) = \begin{cases} \text{ID}, & s(\mathbf{x}) \geq \tau, \\ \text{OOD}, & s(\mathbf{x}) < \tau, \end{cases} \quad (17)$$

where τ is chosen on a validation set (e.g., to control the false positive rate on ID data).

Theoretical Questions

Question 1

Assume that the in-distribution (ID) data are modeled using a single-component Gaussian model:

$$p_{\text{ID}}(x) = \mathcal{N}(x; \mu, \Sigma).$$

1. Write the log-likelihood function $\log p_{ID}(x)$ explicitly.
2. Show that the condition

$$\log p_{ID}(x) \geq \tau$$

is equivalent to x lying inside or on an ellipsoid in the feature space.

3. Derive the exact equation of this ellipsoid in terms of μ , Σ , and τ .

Hint: Relate likelihood thresholding to the Mahalanobis distance.

Question 2

Consider a Gaussian Mixture Model with K components and parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$. For a given sample x :

1. Show that the log-likelihood

$$\log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x; \mu_k, \Sigma_k) \right)$$

cannot be written as a sum of the log-likelihoods of individual components.

2. Using Jensen's inequality, derive a lower bound on the above log-likelihood.
3. Explain how this lower bound is related to the objective function optimized by the EM algorithm.

Question 3

During the M-step of the EM algorithm, the GMM parameters are updated.

1. Show that the update rule for the means,

$$\mu_k^{(t+1)} = \frac{1}{N_k^{(t)}} \sum_{i=1}^n \gamma_{ik}^{(t)} x_i,$$

can be obtained by maximizing the expected complete-data log-likelihood with respect to μ_k .

2. Compute the gradient of the objective function with respect to μ_k and set it equal to zero.
3. Explain why this update corresponds to a weighted average of the data points.

Question 4

Assume that the ID data are drawn from a d -dimensional Gaussian distribution with isotropic covariance $\Sigma = \sigma^2 I$.

1. Show that the expected log-likelihood of an ID sample scales with the dimension d as:

$$\mathbb{E}[\log p(x)] = -\frac{d}{2} \log(2\pi\sigma^2) - \frac{d}{2}.$$

2. Now assume that an OOD sample with a large norm but regular structure is observed. Using mathematical reasoning, explain how it is possible that

$$\log p_{ID}(x_{\text{OOD}}) > \log p_{ID}(x_{\text{ID}}).$$

3. Based on this analysis, explain mathematically why likelihood alone is not a fully reliable criterion for OOD detection, especially in high-dimensional spaces.